

딥러닝 · LLM · 최적화 정리

LG Aimers 온라인 교육에서 배운 이론을 정리하고, 해커톤 경험과 연결한 기록

딥러닝 기초 · 트랜스포머 · LLM · 에이전트 · 블록 최적화 · 의사결정 중심 학습 · 시계열 · 2026

이 자료의 범위

「테이블 머신러닝 정리」의 자매편이다. 그쪽이 대회에서 직접 쓴 것을 다룬다면, 이 자료는 교육에서 배웠지만 이번 대회에는 직접 쓰지 않은 영역을 정리한 것이다.

다만 8부에서 두 자료를 잇는다 — 여기서 배운 개념 여럿이 대회에서 겪은 일을 사후에 설명해 주었다. 특히 6부 의사결정 중심 학습은 "예측을 잘하는 것과 좋은 결정을 내리는 것은 다르다"는 원리를 다루는데, 그것이 대회에서 부딪힌 문제와 정확히 같은 구조였다.

범위: 딥러닝·NLP 1·3·4·5강 / 최적화 1·3강 / 시계열 4강. 나머지 강의(수학 기초, LLM 평가 등)는 이 정리에 포함하지 않았다.

목차

1부 · 딥러닝 기초

- 1.1 AI · 머신러닝 · 딥러닝
- 1.2 학습의 네 가지 방식
- 1.3 신경망과 역전파
- 1.4 XOR 문제와 비선형성

2부 · 트랜스포머

- 2.1 Seq2Seq 와 병목
- 2.2 어텐션
- 2.3 셀프 어텐션과 트랜스포머
- 2.4 생성 탐색 알고리즘

3부 · LLM

- 3.1 사전학습과 미세조정
- 3.2 인컨텍스트 러닝
- 3.3 정렬 문제와 RLHF
- 3.4 RAG

4부 · LLM 에이전트

4.1 에이전트 구성

4.2 메모리

4.3 도구와 MCP

4.4 ReAct 와 Reflexion

4.5 멀티 에이전트

5부 · 최적화

5.1 최적화 문제의 형태

5.2 블록 최적화

5.3 문제 수식화의 중요성

6부 · 의사결정 중심 학습

6.1 predict-then-optimize 의 한계

6.2 DFL

6.3 손실 결합과 충돌

7부 · 시계열

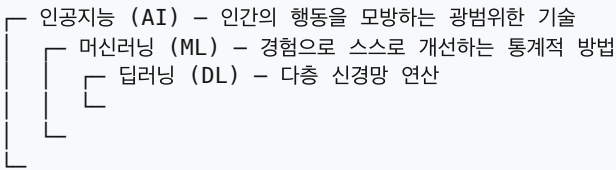
7.1 i.i.d 가정의 붕괴

7.2 두 접근법

8부 · 해커톤과의 연결

1부 · 딥러닝 기초

1.1 AI · 머신러닝 · 딥러닝의 관계



학습의 정의 (Tom Mitchell) "작업 T와 성능 지표 P에 대해, 경험 E를 통해 P로 측정된 T의 성능이 향상되면 그 프로그램은 E로부터 학습한다."

손글씨 인식 예: T=숫자 분류, P=정확도, E=라벨이 달린 이미지 데이터셋.

왜 이 정의가 실전에서 중요한가 P를 무엇으로 잡느냐가 모든 것을 결정한다. 대회에서 P는 Brier였고, 그래서 "정확도"가 아니라 **확률의 정확성**을 올려야 했다. 이 구분을 놓치면 엉뚱한 것을 최적화한다 — 6부의 주제이기도 하다.

1.2 학습의 네 가지 방식

방식	피드백	예
지도 학습	정답(라벨)이 주어진다	스팸 분류, 회귀·분류
비지도 학습	라벨 없이 구조·패턴을 찾는다	군집화(k-Means), 데이터 압축
반지도 학습	적은 라벨 + 많은 무라벨	가상 라벨(pseudo-label) 후 재학습
강화 학습	지연된 보상. 훈련 데이터가 따로 없다	체스 AI, 로봇 제어

강화 학습은 **에이전트**가 **환경**과 상호작용하며 시행착오로 최적 **행동**을 찾는다. 이 구조가 4부의 LLM 에이전트와 3.3의 RLHF로 다시 나온다.

1.3 신경망과 역전파

층	하는 일
입력층	데이터의 수치적 표현
은닉층	(입력 × 가중치)의 합 + 편향 → 활성화 함수(비선형) 통과
출력층	예측값 반환 (회귀=연속값, 분류=클래스 확률)

학습은 세 단계의 반복이다

- 순전파 입력 → 출력, '예측'을 만든다
- 오차 계산 예측과 정답의 차이 = 손실(Loss)
- 역전파 출력→입력 방향으로 오차를 되돌리며,
각 가중치가 오차에 기여한 정도를 기울기로 계산하고
손실이 줄어드는 방향으로 가중치를 조정

테이블 데이터와의 연결 GBDT도 "잔차를 보고 다음 트리를 더한다"는 점에서 같은 사고방식이다 (자매편 0.6). 다만 GBDT는 함수를 더해 나가고, 신경망은 가중치를 고쳐 나간다.

1.4 XOR 문제와 비선형 활성화 함수

직관 선형 분류기는 직선 하나로 가를 수 있는 문제만 푼다. XOR은 직선으로 못 가른다 — 민스키와 페퍼트가 단일 퍼셉트론으로 불가능함을 증명했다.

핵심: 선형 유닛만 쌓으면 아무리 깊어도 결국 하나의 선형 함수가 된다. 층을 늘리는 것이 의미를 가지려면 사이에 **비선형 함수**가 있어야 한다. 2층 ReLU면 XOR이 풀린다.

활성화 함수	형태	비고
Sigmoid	0~1로 압축	확률 출력에 쓰이나 기울기 소실
Tanh	-1~1	Sigmoid보다 중심이 0
ReLU	$\max(0, x)$	계산이 단순하고 기울기 소실이 적다
Softmax	로짓 → 합이 1인 확률분포	분류 최종 층. 손실 계산의 전제

2부 · 트랜스포머와 어텐션

언어 모델링의 진화: Seq2Seq(RNN) → Seq2Seq + Attention → Transformer

2.1 Seq2Seq 와 병목 문제

직관 인코더 RNN이 문장 전체를 벡터 하나로 압축하고, 디코더가 그것만 보고 번역한다. 문제는 명백하다 — 긴 문장을 벡터 하나에 담을 수 없다. 디코더는 마지막 은닉 상태에만 의존하므로 문장 앞부분 정보가 사라진다.

2.2 어텐션 메커니즘

직관 "압축하지 말고, 필요할 때마다 원문을 다시 보자." 디코더가 단어를 하나 만들 때마다 소스 문장의 어디에 주의를 기울일지 결정한다.

- ① 정렬 점수 디코더 현재 상태 s_{i-1} 와 인코더 각 은닉상태 h_j 의 유사도
- ② 가중치 Softmax 정규화 $\rightarrow \alpha_{ij}$
- ③ 컨텍스트 $c_i = \sum_j \alpha_{ij} \cdot h_j$ (인코더 상태들의 가중합)

2.3 셀프 어텐션과 트랜스포머

RNN의 한계 장기 의존성 포착 실패 · 기울기 소실 · 한 번에 하나씩만 처리(병렬화 불가)

셀프 어텐션 어텐션을 입력 문장 자기 자신에게 적용한다. 각 단어가 문장 내 다른 단어들과 얼마나 관련되는지 계산해 문맥이 반영된 새 표현을 만든다 (예: "it"이 무엇을 가리키는지).

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

트랜스포머는 RNN을 완전히 버리고 셀프 어텐션만으로 구성한 모델이다. 병렬 처리가 쉽고, 단어 간 최대 경로 길이가 $O(1)$ 로 줄어 멀리 떨어진 의존성도 잡는다.

구조의 네 요소

요소	왜 필요한가
포지셔널 임베딩	순차 입력이 아니라 순서 정보가 없다. 사인·코사인으로 위치를 임베딩에 더한다
멀티 헤드 어텐션	어텐션을 여러 헤드로 나눠 병렬로 다른 관점을 학습한 뒤 결합
잔차 연결	입력을 출력에 그대로 더해 기울기 소실을 막는다
층 정규화	학습 안정화

인코더는 멀티헤드 어텐션 + FFN을 6층 쌓는다. 디코더는 여기에 인코더-디코더 어텐션(Q는 디코더, K·V는 인코더 출력)이 추가되고, 셀프 어텐션은 미래 단어를 못 보도록 마스킹된다.

2.4 생성 탐색 알고리즘

생성은 자기회귀다 — 다음 단어를 예측하고, 그것을 입력 끝에 붙여 다시 예측한다.

방법	동작	특징
Greedy Search	매 순간 확률 최대 단어 하나	단순, 국소적으로만 최적
Beam Search	상위 N개 경로(num_beams)를 유지	성능이 더 우수

익숙한 구조 Greedy는 결정 트리가 분할을 고르는 방식과 같다(자매편 0.2) — 매 단계 최선을 고르지만 전체 최적은 보장하지 않는다. Beam Search는 그 약점을 후보를 여러 개 들고 가서 완화한 것이다.

3부 · LLM — 사전학습에서 정렬까지

Word2Vec(2013) → Attention → Transformer(2017) → BERT·GPT-1(2018) → GPT-3(2020) → InstructGPT·ChatGPT(2022) → GPT-4·LLaMA(2023) → Claude-3·Gemini(2024~)

3.1 사전학습과 미세조정

Word2Vec의 한계 단어마다 벡터가 하나였다. "영화를 보다"와 "사과를 보다"의 '보다'가 같은 벡터다 — 문맥에 따라 의미가 달라지는 것을 반영하지 못한다.

현대 NLP는 **모델 파라미터 전체**를 대규모 데이터로 사전학습해 초기화한다. 입력의 일부를 가리고 **재구성**하게 학습시키면(다음 단어 예측, 빈칸 맞추기) 그 과정에서 문법·상식·추론 능력이 생긴다.

Step 1. 사전학습 수십억 텍스트로 비지도 학습 → 일반적 언어 표현
Step 2. 미세조정 작은 라벨 데이터 + 출력층 추가 → 특정 작업에 적응

아키텍처	대표 모델	강점
인코더 기반	BERT	이해·분류
인코더-디코더	BART, T5	변환·요약
디코더 기반	GPT	생성

3.2 인컨텍스트 러닝

직관 GPT-3(1750억 파라미터)급에서 **파라미터를 바꾸지 않고 프롬프트만으로** 새 작업을 수행하는 능력이 나타났다. **Zero-shot**(예시 없음) / **Few-shot**(소수 예시)로 요약·번역·코딩을 한 모델이 다 한다.

3.3 정렬 문제와 RLHF

미정렬(Misalignment) GPT-3는 다음 단어를 잘 예측할 뿐, 사용자의 **지시를 따르지**는 않는다. 학습 목표와 우리가 원하는 것이 어긋나 있다.

바람직한 모델의 **3H**: 유익함(Helpful) · 정직함(Honest) · 무해함(Harmless)

RLHF (인간 피드백 기반 강화학습)
① SFT 사람이 쓴 바람직한 답변으로 미세조정
② 보상 모델 사람이 여러 답변에 선호 순위 → 보상 점수 예측 모델 학습
③ 강화학습 언어 모델(에이전트)이 보상 모델의 피드백으로 정책 최적화 (PP0)

ChatGPT는 InstructGPT와 같은 RLHF를 쓰되 **대화 형식** 데이터에 특화한 것이다. **Base 모델**은 사전학습만 되어 텍스트 완성을 잘 하고, **Instruct 모델**은 SFT·RLHF로 지시를 따르도록 튜닝된 것이다.

여기가 6부와 이어진다 "학습 목표(다음 단어 예측)"와 "진짜 원하는 것(지시 이행)"이 다르다는 문제 — 이것이 **6부 의사결정 중심 학습**의 핵심 문제의식과 정확히 같다.

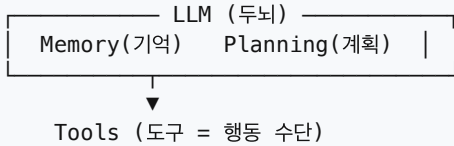
3.4 RAG (검색 증강 생성)

직관 모델의 지식은 학습 시점에 멈춰 있다. 최신 정보를 모르고, 모르는 것을 지어낸다(환각). 그러면 답을 만들기 전에 외부 문서를 찾아와 참고하게 하면 된다.

4부 · LLM 에이전트

4.1 에이전트란

직관 LLM 자체는 최신 정보 접근과 복잡한 연산에 약하다. **외부 도구와 연결해 능력을 확장한 것이 에이전트(Augmented LLM)**다. 센서로 환경을 인지하고, 도구로 행동하고, LLM으로 추론한다.



4.2 메모리

종류	방식	한계
단기 기억	대화 내역을 프롬프트에 담는다	컨텍스트 윈도우 크기 제한 → 길어지면 요약본으로 대체
장기 기억	외부 벡터 DB에 임베딩으로 저장, 필요한 것만 RAG로 검색	검색 품질에 의존

4.3 도구 사용과 MCP

Function Calling: LLM이 어떤 도구를 언제 호출할지 JSON으로 스스로 생성한다.

Toolformer: 어떤 API를 언제 부를지 **모델이 스스로 결정하도록 훈련된 모델**.

MCP (Model Context Protocol) 수많은 API 연동을 표준화하기 위한 규격. **MCP Server**가 도구·컨텍스트를 제공하고(GitHub, 날씨 등), **MCP Client/Host**가 연결을 관리한다(예: IDE).

4.4 계획과 추론 — ReAct, Reflexion

직관 복잡한 일을 하려면 **대답하기 전에 생각**해야 한다(Chain-of-Thought). 그리고 생각만으로는 부족하다 — **행동하고 결과를 보고 다시 생각**해야 한다.

ReAct 순환

- | | |
|--------------------------|-----------|
| ① Thought (생각) | 현재 상황을 추론 |
| ② Action (행동) | 도구 선택·호출 |
| ③ Observation (관찰) | 결과 분석 |
| └───────── 반복 ─────────┘ | |

Reflexion: ReAct에 **평가자와 자아 성찰** 모듈을 더해 **이전 실패로부터 배우게** 하는 기법.

4.5 멀티 에이전트 협업

감독관(Supervisor) 에이전트가 전문화된 여러 에이전트에게 업무를 할당하고 통신을 조율한다. 핵심 과제는 두 가지 — **각 에이전트를 어떻게 만들 것인가(Initialization)**와 **어떻게 조율할 것인가(Orchestration)**.

이건 실제로 해봤다 해커톤에서 AI 4종을 병렬로 운용했다. 교육에서 배운 조울 문제가 그대로 나타났고, 실전에서 얻은 규칙은 8부에 정리했다.

5부 · 최적화

5.1 최적화 문제의 형태

minimize $f_0(x)$ f_0 : 목적 함수
subject to $f_i(x) \leq b_i, \quad i = 1, \dots, m$ f_i : 제약 함수
 x : 최적화 변수

포트폴리오 배분(리스크 최소화), 회로 소자 크기(전력 최소화), 데이터 피팅(예측 오차 최소화) — 형태는 모두 같다.

일반 최적화는 어렵다 해를 구하기가 매우 어려워 완벽한 해를 포기하는 타협이 필요하다. 효율적으로 풀리는 것은 사실상 세 범주뿐이다: 최소제곱 · 선형계획 · 볼록 최적화.

5.2 볼록 최적화 — 왜 특별한가

직관 일반 최적화는 봉우리가 여럿이라 어느 것이 진짜 최고인지 알 수 없다. 볼록 문제는 봉우리가 하나뿐이다 — 가짜 최적점이 없으니 기울기를 따라가면 반드시 전역 최적해에 도달한다.

문제	수식	특징
최소제곱	$\min \ Ax - b\ ^2$	해석적 해 $x^* = (A^T A)^{-1} A^T b$ 존재. 완전히 성숙한 기술
선형계획(LP)	$\min c^T x, \text{ s.t. } a_i^T x \leq b_i$	해석적 해는 없으나 연산이 빠르다
볼록 최적화	목적·제약이 모두 볼록	젠센 부등식 $f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$ 만족

자매편과의 연결 Brier 지표가 예측 벡터의 2차형식이라는 것은 곧 볼록이라는 뜻이다. 그래서 블렌드 가중치 최적화가 전역 최적을 보장하고, "섞어보지 않고 계산으로 답을 낸다"가 가능했다. 지표가 볼록인지 확인하는 것이 대회 첫날 할 일인 이유가 여기 있다.

5.3 문제 수식화의 중요성 — 조도 최적화 예시

램프 전력을 조절해 원하는 조도를 맞추는 문제. 근사 해법(전부 같은 전력, 최소제곱 후 반올림)은 비효율적이다. 그런데 볼록 함수 $h(u) = \max(u, 1/u)$ 를 쓰면 완전히 동치인 볼록 문제로 바꿀 수 있고, 정확한 최적해가 나온다.

제약 하나가 난이도를 바꾼다

"램프 10개의 전력 합 제한" → 선형 제약, 쉽게 풀린다

"켜지는 램프 개수 제한" → 이산 변수가 되어 NP-hard

→ 같은 문제도 어떻게 쓰느냐에 따라 풀리고 안 풀린다.

6부 · 의사결정 중심 학습 (DFL)

이 자료에서 가장 실전과 맞닿은 장이다. 8부와 함께 읽기를 권한다.

6.1 predict-then-optimize 의 한계

기존 파이프라인: 데이터 → 예측 모델(ML) → 최적화(Opt) → 의사결정

핵심 문제: 예측 오차(MSE)를 최소화하는 것이 항상 최적의 의사결정으로 이어지지 않는다.

- 재고 관리 — 재고가 남는 오차보다 부족한 오차가 훨씬 치명적이다. 그런데 MSE는 둘을 똑같이 센다.
- 제약 구조 — 해가 최적화 경계에 걸려 있으면, 오차의 크기보다 방향이 결정을 바꾼다.

6.2 DFL — 결정 손실을 직접 최소화한다

$\hat{\theta} = \arg \min_{\theta} \sum_i L_{\text{dec}}(f_{\theta}(x_i), y_i)$ ← 예측 손실이 아니라 ****결정**** 손실

기술적 난제 최적화 단계 $a^*(y)$ 의 그래디언트를 구할 수 없거나 불연속이다. 그래서 대리 목적함수(SPO+, Linear Reparameterization, LODL 등)를 쓴다.

접공간 투영법 (PEAR) 예측 오차의 그래디언트를 활성 제약의 접공간에 투영해 실제로 결정에 영향을 주는 오차만 남긴다. SPO+ 대비 연산이 빠르고 Regret 지표가 크게 개선된다.

실제 연구 결과가 시사하는 것 포트폴리오 선택에 DFL을 적용하면 MSE는 오히려 나빠지는데 실전 성과(샤프 지수)는 향상된다. 편입할 자산은 과대평가하고 제외할 자산은 과소평가해 결정을 명확히 하기 때문이다.

→ "예측 지표가 좋아지는 것"과 "결과가 좋아지는 것"은 별개다.

6.3 손실 결합과 그래디언트 충돌

$L_{\text{combined}} = L_{\text{pred}} + \alpha \cdot L_{\text{dec}}$ (실무에서는 $\alpha = 0.25 \sim 0.75$ 가 최적)

두 가지 문제 과적합 — 결정에만 치중하면 예측 모델 본래의 표현력이 저하된다.

충돌 — 두 그래디언트의 방향이 반대이고 크기 차이가 커서 단순 합산하면 학습이 한쪽으로 쏠린다.

7부 · 시계열 분석

7.1 i.i.d 가정의 붕괴

일반 통계·머신러닝은 데이터가 서로 독립이라 가정한다. 시계열은 인접 시점끼리 관계가 있어 그 가정이 깨진다.

→ 그래서 무작위 분할 교차검증을 쓰면 안 된다. 미래 정보가 과거 학습에 새어 들어간다.

7.2 두 접근법

방식	특징	장점	예
Model-driven	수식·분포 가정이 미리 정해져 있다	도메인 지식 활용이 쉽고 일반화·해석이 수월	ARIMA, GARCH
Data-driven	분포에 대한 특별한 가정이 없다	복잡·비선형 관계를 잘 반영	Random Forest, 신경망

대회에서 택한 것 데이터가 시준 단위로 나뉘어 있었고, 문제는 시간 외삽이었다. Data-driven(GBDT)을 쓰되 검증만은 시계열 규율을 따랐다 — 무작위 분할이 아니라 전방 폴드로만 평가했다.

8부 · 해커톤과의 연결

교육에서 배운 개념 중 대회에서 겪은 일을 사후에 설명해 준 것들이다. 자매편 「테이블 머신러닝 정리」의 해당 절을 함께 보면 좋다.

① DFL ↔ "단독 성능이 좋은 모델이 블렌드에 도움이 안 되는 이유"

DFL의 문제의식은 "예측 오차 최소화 ≠ 최적 의사결정"이다. 대회에서 부딪힌 것은 "단독 성능 최대화 ≠ 블렌드 기여 최대화"였다. 구조가 같다 — 중간 지표를 최적화하면 최종 목적에서 멀어질 수 있다.

DFL	대회
MSE를 낮춰도 결정이 나빠질 수 있다	단독 성능을 올려도 블렌드 기여가 0이 될 수 있다
실측: MSE는 나빠지는데 샤프 지수는 향상	실측: 단독 성능 최고 모델의 최적 가중치가 0.000
진짜 목적함수는 결정 손실	진짜 목적함수는 p (잔차와 고유방향의 상관)

교훈: 무엇을 최적화하고 있는지 항상 확인하라. "상관을 낮추자"를 목표로 삼으면 노이즈 주입으로 달성되면서 점수가 떨어지는 것도 같은 함정이다.

② 그래디언트 충돌 ↔ "한 번에 하나만 바꿔라"

DFL에서 L_{pred} 와 L_{dec} 를 합치면 방향이 반대이고 크기가 달라 학습이 쏠린다.

대회에서도 두 축을 동시에 바꾼 제출은 결과 해석이 불가능했다 — 점수가 내려갔는데 어느 쪽 탓인지 알 수 없어 슬롯을 쓰고도 배운 게 없었다. 이후 규칙을 "한 번에 하나만"으로 고정했다.

③ 볼록 최적화 ↔ "왜 블렌드를 계산으로 풀 수 있었나"

볼록 문제는 봉우리가 하나라 기울기를 따라가면 전역 최적에 닿는다. Brier 지표가 예측 벡터의 2차형식(= 볼록)이었기 때문에 모델을 실제로 섞어보지 않고 최적 가중치를 계산할 수 있었다.

반대로 지표가 AUC였다면 이 접근은 통하지 않았다. 대회 첫날 지표의 대수적 성질을 확인하라는 규칙이 여기서 나온다.

④ 시계열의 i.i.d 붕괴 ↔ "전방 폴드"

시계열은 독립 가정이 깨지므로 무작위 분할이 위험하다. 대회 데이터도 시즌 단위였고, 실제로 예측 시계(몇 년 뒤를 맞히는가)가 폴드와 배포에서 다르면 로컬 값이 가치를 과대평가한다는 것을 실측으로 확인했다 (1년 격차에서 +33.80이던 모델이 2년 격차에서 +0.00).

⑤ 멀티 에이전트 조율 ↔ AI 4종 병렬 운용

교육에서 배운 과제는 **Agent Initialization**과 **Agent Orchestration**이었다. 실제로 AI 4종을 굴리며 얻은 규칙은 이렇다:

1. **실행자와 판정자를 분리한다** — 자기 결과를 자기가 판정하면 낙관 편향이 걸린다
2. **SSOT 파일 하나에 모든 판정을 기록해 중복 실험을 막는다**
3. **판정 기준을 코드로 고정한다** (사람 판단 개입 최소화)
4. **산출물 접수 규격을 미리 정한다** — 검증 점수 없는 산출물은 접수하지 않는다
5. 에이전트 리포트는 **기본적으로 미검증**으로 취급하고, 직접 재현한 수치만 인용한다

5번이 특히 중요했다. 실제로 **존재하지 않는 점수를 "공식 보고서" 형식으로 작성한 산출물**이 나온 적이 있고, 한 번에 111건을 격리했다. Reflexion이 말하는 **자아 성찰·평가자 모듈**이 왜 필요한지를 실감한 사례다.

⑥ 트랜스포머·LLM은 왜 이 대회에 안 썼나

테이블 데이터에서는 **GBDT가 여전히 강하다**. 실제로 신경망 **한 종**을 블렌드에 넣는 것은 크게 성공했지만(+36.2), **종류를 늘리는 것은** 일관되게 손해였다(-24 ~ -39). 신경망끼리는 서로 상관이 높아 두 번째부터 기여가 0이기 때문이다.

다만 강의에서 본 비교표(TabPFN·TabM 등이 XGBoost와 경쟁)는 **이 판도가 바뀌고 있음**을 보여준다. 다음 대회에서 확인할 지점이다.